



Ανάπτυξη και Ασφάλεια Πληροφοριακών Συστημάτων

Συστήματα Ανάλυσης & Διαχείρισης Μεγάλων Δεδομένων

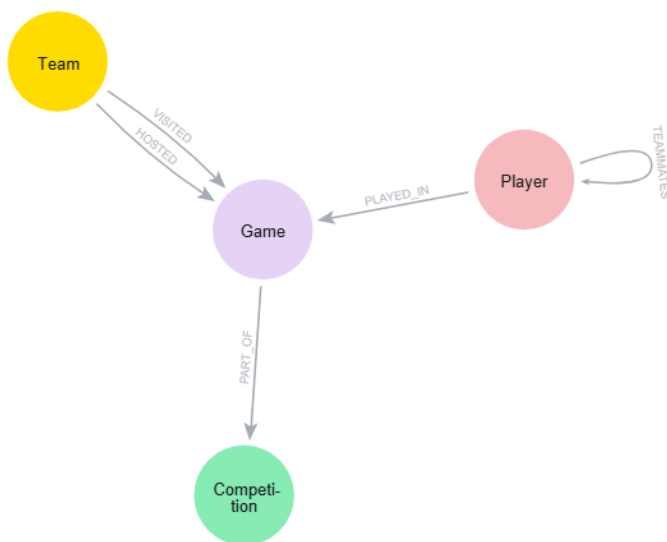
Εργασία 2 - Neo4j

2025-2026

Ελένη Κεχριώτη

f3312503

Η προσέγγιση που ακολουθήθηκε για τη μοντελοποίηση των δεδομένων βασίστηκε στις αρχές των **Property Graphs**, με στόχο την εξάλειψη της πλεονάζουσας πληροφορίας και την ταχύτερη εκτέλεση των ερωτημάτων Cypher . Για τον σκοπό αυτό, τα δεδομένα του αρχείου *FootBallData.csv* διαχωρίστηκαν σε τέσσερις **βασικές οντότητες (nodes)**:



- 1) **Player**: Περιλαμβάνει τα μόνιμα χαρακτηριστικά του παίκτη, όπως το ονοματεπώνυμο, τη θέση και την ημερομηνία γέννησης .
- 2) **Team**: Αντιπροσωπεύει τις ποδοσφαιρικές ομάδες.
- 3) **Competition**: Περιέχει την ονομασία του πρωταθλήματος και τη χώρα διεξαγωγής.
- 4) **Game**: Λειτουργεί ως το κεντρικό σημείο αναφοράς για κάθε αναμέτρηση, αποθηκεύοντας το σκορ, την ημερομηνία και τον νικητή.

Οι σχέσεις μεταξύ των οντοτήτων σχεδιάστηκαν έτσι ώστε να αντικατοπτρίζουν τη δυναμική φύση του αθλήματος.

Η σχέση **PLAYED_IN** συνδέει τον παίκτη με τον αγώνα και είναι ιδιαίτερα σημαντική, καθώς φέρει ιδιότητες (*properties*) που μεταβάλλονται σε κάθε παιχνίδι, όπως τα λεπτά συμμετοχής, τα γκολ, οι ασίστ και οι κάρτες. Οι ομάδες συνδέονται με τους αγώνες μέσω των σχέσεων **HOSTED** και **VISITED**, επιτρέποντας τον σαφή διαχωρισμό γηπεδούχου και φιλοξενούμενου. Αυτή η δομή επιτρέπει την εύκολη διαχείριση περιπτώσεων μεταγραφών, καθώς ένας παίκτης μπορεί να συνδέεται με διαφορετικές ομάδες σε διαφορετικούς αγώνες μέσω της ιδιότητας της ομάδας στη σχέση συμμετοχής του.

Σημείωση: Κατά τον σχεδιασμό του γράφου, αποφεύχθηκε η δημιουργία απευθείας σχέσης μεταξύ των κόμβων **Player** και **Team**. Μια τέτοια σχέση κρίθηκε εννοιολογικά μη δικαιολογημένη για το συγκεκριμένο πρόβλημα, καθώς η συμμετοχή ενός παίκτη σε μια ομάδα ορίζεται δυναμικά ανά αγώνα και ένας παίκτης μπορεί να αγωνιστεί σε έως δύο ομάδες ανά σεζόν. Η πληροφορία της ομάδας αποθηκεύεται ως ιδιότητα στη σχέση συμμετοχής, διασφαλίζοντας την εννοιολογική ορθότητα και την αποφυγή πλεονασμού δεδομένων.

Υλοποίηση

Η υλοποίηση της βάσης έγινε στο περιβάλλον Neo4j με τη χρήση της γλώσσας Cypher . Πριν από την εισαγωγή των δεδομένων, κρίθηκε απαραίτητη η δημιουργία Constraints μοναδικότητας για τα πεδία **PlayerID**, **TeamName**, **GameID** και **CompetitionName** ώστε να διασφαλιστεί η ακεραιότητα των δεδομένων και να αποφευχθεί η δημιουργία πολλαπλών κόμβων για την ίδια οντότητα.

```

CREATE CONSTRAINT PlayerIDConstraint IF NOT EXISTS FOR (p:Player)
REQUIRE p.playerID IS UNIQUE;
CREATE CONSTRAINT TeamNameConstraint IF NOT EXISTS FOR (t:Team)
REQUIRE t.teamName IS UNIQUE;
CREATE CONSTRAINT GameIDConstraint IF NOT EXISTS FOR (g:Game) REQUIRE
g.gameID IS UNIQUE;
CREATE CONSTRAINT CompetitionNameConstraint IF NOT EXISTS FOR
(c:Competition) REQUIRE c.name IS UNIQUE;

```

Η εισαγωγή πραγματοποιήθηκε με την εντολή **LOAD CSV**, χρησιμοποιώντας ως διαχωριστή τον χαρακτήρα ; .

```

LOAD CSV WITH HEADERS FROM
'https://www.dropbox.com/scl/fi/qsm1wgnevo9a7dtp923r7/FootBallData.csv?rlkey=cbytm7qhv4blsuju4zd4lpoyl&st=xtql3jaq&dl=1' AS row
FIELDTERMINATOR ';'

```

Για τη φόρτωση χρησιμοποιήθηκε η στρατηγική **MERGE**, η οποία λειτουργεί ως συνδυασμός αναζήτησης και δημιουργίας. Με αυτόν τον τρόπο, κατά την ανάγνωση των **98.751** εγγραφών, το σύστημα δημιούργησε νέους κόμβους μόνο όταν δεν προϋπήρχαν, ενώ σε αντίθετη περίπτωση ενημέρωνε τις υπάρχουσες οντότητες ή δημιουργούσε τις απαραίτητες σχέσεις. Η διαδικασία χωρίστηκε σε δύο στάδια: αρχικά τη δημιουργία των κόμβων

```

// Δημιουργία Παικτών
MERGE (p:Player {playerID: toInteger(row.PlayerID)})
ON CREATE SET p.name = row.PlayerName,
p.position = row.Position,
p.birthDate = row.BirthDate

```

```

// Δημιουργία Ομάδων
MERGE (t:Team {teamName: row.PlayerTeamName})

```

```

// Δημιουργία Πρωταθλημάτων
MERGE (c:Competition {name: row.CompetitionName})
ON CREATE SET c.country = row.Country

```

```

// Δημιουργία Αγώνων
MERGE (g:Game {gameID: toInteger(row.GameID)})
ON CREATE SET g.date = row.GameDate,
g.homeGoals = toInteger(row.HomeGoals),
g.awayGoals = toInteger(row.AwayGoals),
g.winner = toInteger(row.Winner);

```

και στη συνέχεια τη δημιουργία των συσχετίσεων, εξασφαλίζοντας έτσι τη βέλτιστη χρήση των πόρων του συστήματος.

```
MATCH (p:Player {playerID: toInteger(row.PlayerID)})
MATCH (g:Game {gameID: toInteger(row.GameID)})
MATCH (c:Competition {name: row.CompetitionName})
MATCH (h:Team {teamName: row.HomeTeamName})
MATCH (a:Team {teamName: row.AwayTeamName})

// Σχέση συμμετοχής: Εδώ αποθηκεύεται η ομάδα του παίκτη για να
υποστηρίζονται οι μεταγραφές [cite: 46, 49]
MERGE (p)-[r:PLAYED_IN]->(g)
ON CREATE SET r.team = row.PlayerTeamName,
r.minutes = toInteger(row.Minutes),
r.goals = toInteger(row.Goals),
r.assists = toInteger(row.Assists),
r.yellowCards = toInteger(row.YellowCards),
r.redCards = toInteger(row.RedCards)

// Σύνδεση αγώνα με το πρωτάθλημά του
MERGE (g)-[:PART_OF]->(c)

// Σύνδεση των ομάδων που έπαιξαν στον αγώνα
(Γηπεδούχος/Φιλοξενούμενη)
MERGE (h)-[:HOSTED]->(g)
MERGE (a)-[:VISITED]->(g);
```

✓ Created 108,864 relationships, set 493,755 properties

Completed after 29,248 ms

Τα παραπάνω έτρεξαν στο *Neo4j sandbox* και δημιουργήθηκαν **9.841** κόμβοι και **108.864** συσχετίσεις.

Δημιουργήθηκε μια πρόσθετη σχέση που συνδέει παίκτες που συνυπήρξαν στην ίδια ομάδα στον ίδιο αγώνα, αποθηκεύοντας τον αριθμό των κοινών συμμετοχών τους. Αυτή η σχέση είναι απαραίτητη για την απάντηση των ερωτημάτων 6 και 7.

```
MATCH
(p1:Player)-[r1:PLAYED_IN]->(g:Game)<-[r2:PLAYED_IN]-(p2:Player)
WHERE p1.playerID < p2.playerID AND r1.team = r2.team
MERGE (p1)-[rel:TEAMMATES]->(p2)
ON CREATE SET rel.games_together = 1
ON MATCH SET rel.games_together = rel.games_together + 1;
```

✓ Created 71,724 relationships, set 679,856 properties

Completed after 24,913 ms

Επερωτήματα

Σε αυτή την ενότητα παρουσιάζονται τα επερωτήματα που υλοποιήθηκαν για την άντληση στατιστικών στοιχείων από τον γράφο. Κάθε ερώτημα σχεδιάστηκε με γνώμονα την αποδοτική πλοήγηση στις σχέσεις του γράφου.

1. Στατιστικά Νικών και Ισοπαλιών ανά Πρωτάθλημα

Το ερώτημα υπολογίζει το σύνολο των εντός έδρας νικών, εκτός έδρας νικών και ισοπαλιών για κάθε πρωτάθλημα, ταξινομώντας τα αποτελέσματα αλφαβητικά. Παρακάτω φαίνεται ο κώδικας που εκτελέστηκε και ένα screenshot από τα αποτελέσματα.

```
MATCH (c:Competition)<-[:PART_OF]-(g:Game)
RETURN c.name AS Competition_Name,
       SUM(CASE WHEN g.winner = 1 THEN 1 ELSE 0 END) AS Home_Wins,
       SUM(CASE WHEN g.winner = 2 THEN 1 ELSE 0 END) AS Away_Wins,
       SUM(CASE WHEN g.winner = 0 THEN 1 ELSE 0 END) AS Draws
ORDER BY Competition_Name ASC
```



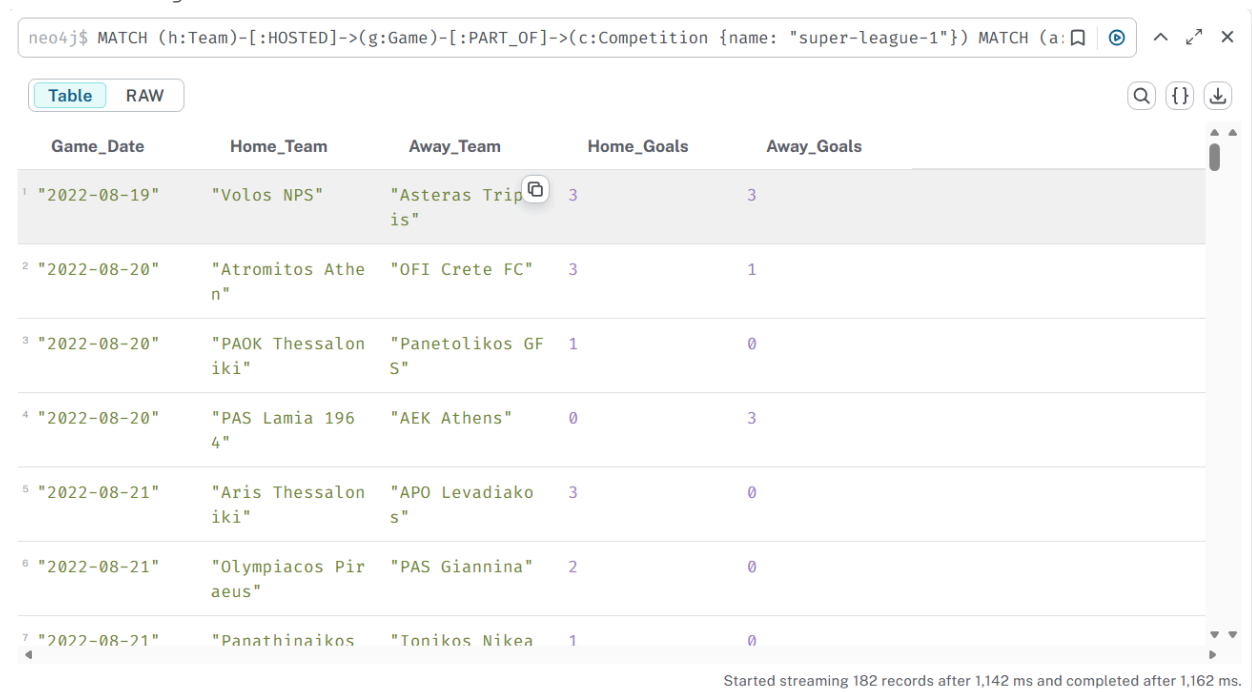
The screenshot shows a Neo4j query result interface. At the top, the Cypher query is entered in a text box. Below the query, there are tabs for 'Table' and 'RAW', with 'Table' selected. To the right of the tabs are icons for search, JSON, and download. The main area displays a table with 4 columns: 'Competition_Nam', 'Home_Wins', 'Away_Wins', and 'Draws'. The table contains 8 rows of data, each representing a different football league. The rows are numbered 1 through 8 on the left. At the bottom of the interface, a status bar indicates 'Started streaming 14 records after 520 ms and completed after 720 ms.'

	Competition_Nam	Home_Wins	Away_Wins	Draws
1	"bundesliga"	116	72	64
2	"eredivisie"	112	82	66
3	"jupiler-pro-league"	133	98	66
4	"laliga"	129	89	71
5	"liga-portugal-bwin"	119	86	47
6	"ligue-1"	131	101	78
7	"premier-league"	142	93	71
8	"premier-liga"	82	60	42

2. Αποτελέσματα Αγώνων Ελληνικού Πρωταθλήματος (Super League 1)

Εμφανίζονται οι ημερομηνίες, οι ομάδες και τα σκορ όλων των αγώνων του ελληνικού πρωταθλήματος, ταξινομημένα χρονολογικά. Παρακάτω φαίνεται ο κώδικας που εκτελέστηκε και ένα screenshot από τα αποτελέσματα.

```
MATCH (h:Team)-[:HOSTED]->(g:Game)-[:PART_OF]->(c:Competition {name:
"super-league-1"})
MATCH (a:Team)-[:VISITED]->(g)
RETURN g.date AS Game_Date,
       h.teamName AS Home_Team,
       a.teamName AS Away_Team,
       g.homeGoals AS Home_Goals,
       g.awayGoals AS Away_Goals
ORDER BY g.date ASC
```



neo4j\$ MATCH (h:Team)-[:HOSTED]->(g:Game)-[:PART_OF]->(c:Competition {name: "super-league-1"}) MATCH (a:Team)-[:VISITED]->(g) RETURN g.date AS Game_Date, h.teamName AS Home_Team, a.teamName AS Away_Team, g.homeGoals AS Home_Goals, g.awayGoals AS Away_Goals ORDER BY g.date ASC

	Game_Date	Home_Team	Away_Team	Home_Goals	Away_Goals
1	"2022-08-19"	"Volos NPS"	"Asteras Tripolis"	3	3
2	"2022-08-20"	"Atromitos Athina"	"OFI Crete FC"	3	1
3	"2022-08-20"	"PAOK Thessaloniki"	"Panetolikos GF S"	1	0
4	"2022-08-20"	"PAS Lamia 1964"	"AEK Athens"	0	3
5	"2022-08-21"	"Aris Thessaloniki"	"APO Levadiako S"	3	0
6	"2022-08-21"	"Olympiacos Piraeus"	"PAS Giannina"	2	0
7	"2022-08-21"	"Panathinaikos"	"Tonikos Nikea"	1	0

Started streaming 182 records after 1,142 ms and completed after 1,162 ms.

3. Εντοπισμός Μεταγραφών (Ελλάδα ↔ Εξωτερικό)

Βρίσκουμε παίκτες που αγωνίστηκαν τόσο στην ελληνική Super League όσο και σε κάποιο ξένο πρωτάθλημα κατά την ίδια περίοδο. Παρακάτω φαίνεται ο κώδικας που εκτελέστηκε και ένα screenshot από τα αποτελέσματα.

```

MATCH
(p:Player)-[r1:PLAYED_IN]->(g1:Game)-[:PART_OF]->(c1:Competition
{name: "super-league-1"})
MATCH (p)-[r2:PLAYED_IN]->(g2:Game)-[:PART_OF]->(c2:Competition)
WHERE c1 <> c2
RETURN DISTINCT p.name AS Player_Name,
r1.team AS Greek_Team,
r2.team AS Foreign_Team,
c2.name AS Foreign_Competition
ORDER BY Greek_Team ASC

```

neo4j\$ MATCH (p:Player)-[r1:PLAYED_IN]->(g1:Game)-[:PART_OF]->(c1:Competition {name: "super-league-1"})

Table RAW

	Player_Name	Greek_Team	Foreign_Team	Foreign_Competit
1	"Pape Cheikh"	"Aris Thessaloniki"	"Elche CF"	"laliga"
2	"Vladimir Darida"	"Aris Thessaloniki"	"Hertha BSC"	"bundesliga"
3	"Kevin Soni"	"Asteras Tripolis"	"Hatayspor"	"super-lig"
4	"Laurens De Bokk"	"Atromitos Athina"	"SV Zulte Waregem"	"jupiler-pro-league"
5	"Sebastian Gronning"	"OFI Crete FC"	"Aarhus GF"	"superligaen"
6	"Pep Biel"	"Olympiacos Piraeus"	"FC Copenhagen"	"superligaen"
7	"Diadie Samasse"	"Olympiacos Piraeus"	"TSG 1899 Hoffe"	"bundesliga"

Started streaming 21 records after 1,102 ms and completed after 2,103 ms.

4. Πρώτος Σκόρερ ανά Ομάδα (Premier League)

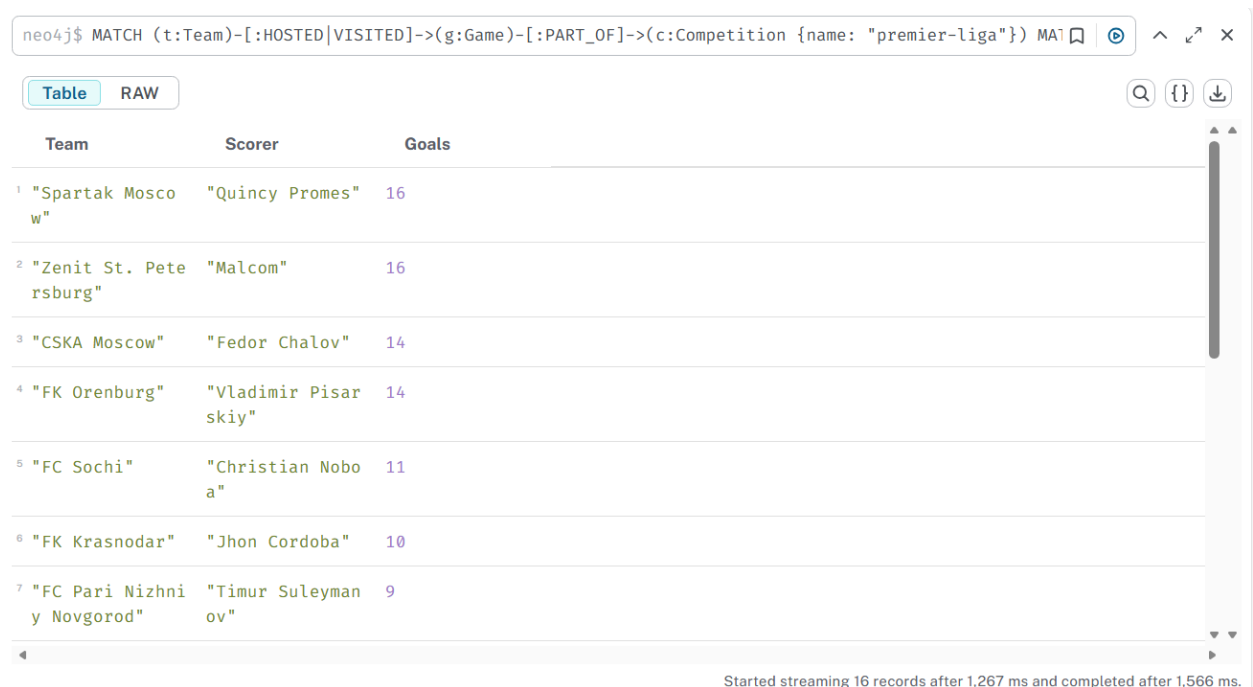
Υπολογίζεται ο κορυφαίος σκόρερ για κάθε ομάδα του πρωταθλήματος της Premier League. Παρακάτω φαίνεται ο κώδικας που εκτελέστηκε και ένα screenshot από τα αποτελέσματα.

```

MATCH (t:Team)-[:HOSTED|VISITED]->(g:Game)-[:PART_OF]->(c:Competition
{name: "premier-liga"})
MATCH (p:Player)-[r:PLAYED_IN {team: t.teamName}]->(g)
WITH t, p, SUM(r.goals) AS Total_Goals
ORDER BY t.teamName, Total_Goals DESC
WITH t, COLLECT({player: p.name, goals: Total_Goals})[0] AS
Top_Scorer
RETURN t.teamName AS Team, Top_Scorer.player AS Scorer,
Top_Scorer.goals AS Goals

```

ORDER BY Goals DESC



The screenshot shows a Neo4j Cypher query interface. The query is: `neo4j$ MATCH (t:Team)-[:HOSTED|VISITED]->(g:Game)-[:PART_OF]->(c:Competition {name: "premier-liga"}) MA1`. The results are displayed in a table with columns: Team, Scorer, and Goals. The table lists the top 7 teams by goals scored in the Premier League.

	Team	Scorer	Goals
1	"Spartak Mosco w"	"Quincy Promes"	16
2	"Zenit St. Pete rsburg"	"Malcom"	16
3	"CSKA Moscow"	"Fedor Chalov"	14
4	"FK Orenburg"	"Vladimir Pisar skiy"	14
5	"FC Sochi"	"Christian Nobo a"	11
6	"FK Krasnodar"	"Jhon Cordoba"	10
7	"FC Pari Nizhni y Novgorod"	"Timur Suleyman ov"	9

Started streaming 16 records after 1,267 ms and completed after 1,566 ms.

5. Οι 10 πιο "Σκληροί" Αμυντικοί

Λίστα με τους 10 αμυντικούς που δέχθηκαν τις περισσότερες κάρτες (κόκκινες και κίτρινες) στο πρωτάθλημα της Premier League. Παρακάτω φαίνεται ο κώδικας που εκτελέστηκε και ένα screenshot από τα αποτελέσματα.

```
MATCH (p:Player {position: "Defender"})-[:PLAYED_IN]->(g:Game)-[:PART_OF]->(c:Competition {name: "premier-liga"})
RETURN p.name AS Player_Name,
       r.team AS Team,
       SUM(r.redCards) AS Total_Red_Cards,
       SUM(r.yellowCards) AS Total_Yellow_Cards
ORDER BY Total_Red_Cards DESC, Total_Yellow_Cards DESC
LIMIT 10
```


neo4j\$ MATCH (p:Player {position: "Defender"})-[r:PLAYED_IN]->(g:Game)-[:PART_OF]->(c:Competition {name: "Premier League"})

Table RAW

	Player_Name	Team	Total_Red_Cards	Total_Yellow_Card
1	"Glenn Bijl"	"Krylya Sovetov Samara"	1	4
2	"Ilya Kutepov"	"Torpedo Moscow"	1	4
3	"Maksim Nenakhov"	"Lokomotiv Moscow"	1	4
4	"Artem Meshchakov"	"FC Sochi"	1	3
5	"Leo Goglichidze"	"Ural Yekaterinburg"	1	3
6	"Mark Mampassi"	"Lokomotiv Moscow"	1	1
7	"Moussa Sissak"	"FC Sochi"	1	1

Started streaming 10 records after 555 ms and completed after 669 ms.

6. Βασικό Επιθετικό Δίδυμο (Ολυμπιακός)

Αξιοποιώντας τη σχέση **TEAMMATES**, εντοπίζουμε τους δύο επιθετικούς που αγωνίστηκαν μαζί στις περισσότερες αναμετρήσεις. Παρακάτω φαίνεται ο κώδικας που εκτελέστηκε και ένα screenshot από τα αποτελέσματα.

```
MATCH (p1:Player {position: "Attack"})-[rel:TEAMMATES]-(p2:Player {position: "Attack"})
MATCH (p1)-[r:PLAYED_IN]->(g:Game)
WHERE r.team = "Olympiacos Piraeus"
RETURN p1.name AS Player_1, p2.name AS Player_2, rel.games_together AS Matches
ORDER BY Matches DESC
LIMIT 1
```

neo4j\$ MATCH (p1:Player {position: "Attack"})-[rel:TEAMMATES]-(p2:Player {position: "Attack"}) MATCH (p1)-[r:PLAYED_IN]->(g:Game) WHERE r.team = "Olympiacos Piraeus" RETURN p1.name AS Player_1, p2.name AS Player_2, rel.games_together AS Matches ORDER BY Matches DESC LIMIT 1

Table RAW

	Player_1	Player_2	Matches
1	"Georgios Masouras"	"Youssef El Arabi"	23

Started streaming 1 record after 595 ms and completed after 1,095 ms.

7. Κομβικοί Παίκτες - Ανάλυση Δικτύου (Ολυμπιακός)

Εντοπισμός των τριών παικτών που έχουν αγωνιστεί με τους περισσότερους διαφορετικούς συμπαίκτες, υποδεικνύοντας τον κεντρικό τους ρόλο στο σχήμα της ομάδας. Παρακάτω φαίνεται ο κώδικας που εκτελέστηκε και ένα screenshot από τα αποτελέσματα.

```
MATCH (p1:Player)-[:TEAMMATES]-(p2:Player)
MATCH (p1)-[r:PLAYED_IN]->(g:Game)
WHERE r.team = "Olympiacos Piraeus"
RETURN p1.name AS Player_Name, COUNT(DISTINCT p2) AS Total_Teammates
ORDER BY Total_Teammates DESC
LIMIT 3
```



The screenshot shows a Neo4j query result interface. At the top, the query is entered in a text box: `neo4j$ MATCH (p1:Player)-[:TEAMMATES]-(p2:Player) MATCH (p1)-[r:PLAYED_IN]->(g:Game) WHERE r.team = "Oly`. Below the query box, there are two tabs: "Table" (selected) and "RAW". To the right of the tabs are icons for search, JSON, and download. The table displays the results of the query, with columns "Player_Name" and "Total_Teammates". The results are sorted in descending order of "Total_Teammates".

	Player_Name	Total_Teammates
1	"Pep Biel"	53
2	"Cedric Bakambu"	46
3	"Diadie Samassekou"	45

Started streaming 3 records after 364 ms and completed after 663 ms.